Advanced Control Systems

Assignment 1: Comprehensive Kinematic Analysis, Inverse Kinematics, and Jacobians of a PRR Manipulator

Student Name: Filippo Segalla | **Student ID:** VR538212

Department of Engineering for Innovative Medicine (DIMI)

October 20, 2026

Abstract

This report presents a rigorous kinematic modeling and validation study for a 3-DOF PRR (Prismatic-Revolute-Revolute) robotic manipulator specified by a custom URDF description (`00001_PRR_zzx FILIPPO SEGALLA.urdf`). The report details the theoretical foundation, physical rationale, and mathematical derivation behind every stage of analysis: the Denavit-Hartenberg (D-H) frame alignment, direct kinematics transformations, analytical closed-form inverse kinematics, and the derivation of both Geometric ($\mathbf{J}_G$) and Analytical ($\mathbf{J}_a$) Jacobians. Each mathematical development is paired with an explanatory rationale answering *what* operation is conducted and *why* it is required in robot kinematics and control.

---

1 System Architecture & Kinematic Conventions

1.1 Description of the PRR Manipulator

What is being done: We model a 3-degree-of-freedom (3-DOF) spatial manipulator featuring a **PRR** architecture:

1. **Joint 1 (Prismatic, q_1):** Translates vertically along the Z_0-axis, raising or lowering the entire arm structure.
2. **Joint 2 (Revolute, q_2):** Rotates about a vertical axis parallel to Z_0, sweeping the arm across the horizontal plane.
3. **Joint 3 (Revolute, q_3):** Rotates about a horizontal axis orthogonal to the second joint, providing elevation and reach in the vertical plane.

Why it is done: Analyzing the kinematic structure before writing equations establishes the boundary conditions for the manipulator's workspace. Identifying joint types (prismatic vs. revolute) dictates the selection of columns in velocity Jacobians and determines whether joint displacement represents a spatial offset (meters) or a rotational angle (radians).

1.2 Denavit-Hartenberg (D-H) Parameter Convention

What is being done: We assign coordinate frames F_0, F_1, F_2, F_3 to each link following the standard Denavit-Hartenberg convention. The relative transformation between frame $i-1$ and frame i is parameterized by four quantities: link length (a_i), link twist (α_i), link offset (d_i), and joint angle (θ_i).

The elementary transformation matrix $\mathbf{A}_i$ is constructed via four consecutive operations:

$$\mathbf{A}_i = \text{Rot}_{z,\theta_i} \cdot \text{Trans}_{z,d_i} \cdot \text{Trans}_{x,a_i} \cdot \text{Rot}_{x,\alpha_i} \tag{1}$$

Carrying out the matrix multiplication yields the explicit standard D-H matrix:

$$\mathbf{A}_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

Why it is done: A general rigid-body transformation in 3D space requires 6 independent parameters (3 translational, 3 rotational). The D-H convention reduces this requirement to 4 parameters per link by enforcing strict geometric constraints (e.g., placing the X_i-axis along the common normal between Z_{i-1} and Z_i). This systematic reduction simplifies symbolic computation, minimizes potential human error in matrix setup, and provides a unified standard across academic and industrial robotics frameworks.

Table 1: Denavit-Hartenberg parameters for the PRR Manipulator.

Link i	**Joint Type**	a_i [m]	α_i [rad]	d_i [m]	θ_i [rad]
1	Prismatic (P)	0	0	q_1	0
2	Revolute (R)	0	$\pi/2$	0.4	q_2
3	Revolute (R)	0.4	0	0	q_3

2 Direct Kinematics & Frame Transformations

2.1 Base and End-Effector Transformations

What is being done: We incorporate two fixed transformation matrices, $\mathbf{T}_0^B$ and $\mathbf{T}_E^3$:

$$\mathbf{T}_0^B = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.65 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T}_E^3 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3}$$

Why it is done: Standard D-H frames are placed based on geometric conveniences that rarely coincide with the actual physical world origin or the physical tool center point (TCP):

- **Base Transformation ($\mathbf{T}_0^B$):** Relates the world coordinate system (e.g., ground level) to D-H Frame 0. The vertical offset 0.65 m represents the fixed pedestal height of the robot base, while the 90° Z-rotation aligns the internal D-H axis with the URDF coordinate system.
- **Tool Transformation ($\mathbf{T}_E^3$):** Re-aligns the axes of D-H Frame 3 with the actual end-effector flange orientation defined in the URDF, ensuring that position and orientation commands correspond to the true physical end-effector.

2.2 Cumulative Kinematic Transformations

What is being done: The cumulative transformation from the base frame to the i-th frame is computed by sequential matrix multiplication:

$$\mathbf{T}_i^0(q) = \mathbf{T}_0^B \cdot \mathbf{A}_1(q_1) \cdots \mathbf{A}_i(q_i) \tag{4}$$

Evaluating these multiplications symbolically yields:

$$\mathbf{T}_1^0(q) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & q_1 + 0.65 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5}$$

$$\mathbf{T}_2^0(q) = \begin{bmatrix} -\sin q_2 & 0 & \cos q_2 & 0 \\ \cos q_2 & 0 & \sin q_2 & 0 \\ 0 & 1 & 0 & q_1 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{6}$$

$$\mathbf{T}_3^0(q) = \begin{bmatrix} -\sin q_2 \cos q_3 & \sin q_2 \sin q_3 & \cos q_2 & -0.4 \sin q_2 \cos q_3 \\ \cos q_2 \cos q_3 & -\sin q_3 \cos q_2 & \sin q_2 & 0.4 \cos q_2 \cos q_3 \\ \sin q_3 & \cos q_3 & 0 & q_1 + 0.4 \sin q_3 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{7}$$

Multiplying $\mathbf{T}_3^0(q)$ by the end-effector transform $\mathbf{T}_E^3$ gives the final Direct Kinematics matrix $\mathbf{T}_E^B(q)$:

$$\mathbf{T}_E^B(q) = \begin{bmatrix} \cos q_2 & -\sin q_2 \cos q_3 & \sin q_2 \sin q_3 & -0.4 \sin q_2 \cos q_3 \\ \sin q_2 & \cos q_2 \cos q_3 & -\sin q_3 \cos q_2 & 0.4 \cos q_2 \cos q_3 \\ 0 & \sin q_3 & \cos q_3 & q_1 + 0.4 \sin q_3 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8}$$

Extracting the 4th column yields the operational position vector $\mathbf{p}_e = [p_x, p_y, p_z]^T$:

$$\mathbf{p}_e(q) = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} = \begin{bmatrix} -0.4 \sin q_2 \cos q_3 \\ 0.4 \cos q_2 \cos q_3 \\ q_1 + 0.4 \sin q_3 + 1.05 \end{bmatrix} \tag{9}$$

Why it is done: Direct Kinematics provides the fundamental mapping $\mathbf{p}_e = f(q)$ necessary for path planning, collision detection, and workspace visualization. Knowing the exact position of every intermediate joint frame ($\mathbf{T}_1^0, \mathbf{T}_2^0, \mathbf{T}_3^0$) is also required later to compute the partial Jacobians and link centers of mass for dynamic modeling.

3 Closed-Form Inverse Kinematics

3.1 Derivation of Joint Variables

What is being done: Inverse Kinematics solves for the joint variables $q = [q_1, q_2, q_3]^T$ given a target Cartesian coordinate $\mathbf{p}_e = [p_x, p_y, p_z]^T$. We derive an exact closed-form analytical solution.

Derivation Steps:

1. **Solving for q_3:** Summing the squares of p_x and p_y:

$$p_x^2+p_y^2 = (-0.4\sin q_2\cos q_3)^2+(0.4\cos q_2\cos q_3)^2 = 0.16\cos^2 q_3(\sin^2 q_2+\cos^2 q_2) = 0.16\cos^2 q_3 \tag{10}$$

Taking the square root gives:

$$\cos q_3 = \frac{\sqrt{p_x^2+p_y^2}}{0.4} \tag{11}$$

The sine component is determined via the Pythagorean identity $\sin q_3 = \pm\sqrt{1-\cos^2 q_3}$.

2. **Solving for q_2:** Dividing $-p_x$ by p_y:

$$\frac{-p_x}{p_y} = \frac{0.4\sin q_2\cos q_3}{0.4\cos q_2\cos q_3} = \frac{\sin q_2}{\cos q_2} = \tan q_2 \implies q_2 = \text{atan2}(-p_x, p_y) \tag{12}$$

3. **Solving for q_1:** Rearranging the expression for p_z:

$$p_z = q_1 + 0.4\sin q_3 + 1.05 \implies q_1 = p_z - 1.05 - 0.4\sin q_3 \tag{13}$$

3.2 Physical Rationale of Multiple Solutions (Elbow-Up vs. Elbow-Down)

What is being done: Due to the sign ambiguity in $\sin q_3 = \pm\sqrt{1-\cos^2 q_3}$, two valid kinematic branches are calculated:

- **Branch 1 (Elbow-Up, $\sin q_3 > 0$):**

$$q_3^{(1)} = \text{atan2}\left(+\sqrt{1-\cos^2 q_3}, \cos q_3\right), \quad q_1^{(1)} = p_z - 1.05 - 0.4\sin q_3^{(1)} \tag{14}$$

- **Branch 2 (Elbow-Down, $\sin q_3 < 0$):**

$$q_3^{(2)} = \text{atan2}\left(-\sqrt{1-\cos^2 q_3}, \cos q_3\right), \quad q_1^{(2)} = p_z - 1.05 - 0.4\sin q_3^{(2)} \tag{15}$$

Why it is done: Analytical closed-form solutions are computationally far superior to numerical iterative solvers (such as Newton-Raphson) because they compute in deterministic execution time ($O(1)$ complexity) and reveal all possible physical configurations. In real-time trajectory control, having access to both branches allows the controller to select the configuration that avoids mechanical joint limits, prevents self-collisions, or minimizes energy consumption.

4 Geometric Jacobian ($\mathbf{J}_G$)

4.1 Mathematical Formulation

What is being done: The Geometric Jacobian $\mathbf{J}_G(q) \in \mathbb{R}^{6\times 3}$ relates joint velocities $\dot{q}$ directly to the linear ($\mathbf{v}_e$) and angular ($\boldsymbol{\omega}_e$) velocities of the end-effector in 3D physical space:

$$\begin{bmatrix}\mathbf{v}_e \\ \boldsymbol{\omega}_e\end{bmatrix} = \mathbf{J}_G(q)\dot{q} = \begin{bmatrix}\mathbf{J}_{v1} & \mathbf{J}_{v2} & \mathbf{J}_{v3} \\ \mathbf{J}_{w1} & \mathbf{J}_{w2} & \mathbf{J}_{w3}\end{bmatrix}\begin{bmatrix}\dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3\end{bmatrix} \tag{16}$$

Columns are derived according to joint type:

- **Prismatic Joint i:** $\mathbf{J}_{vi} = \mathbf{z}_{i-1}$, $\mathbf{J}_{wi} = \mathbf{0}$.
- **Revolute Joint i:** $\mathbf{J}_{vi} = \mathbf{z}_{i-1} \times (\mathbf{p}_e - \mathbf{p}_{i-1})$, $\mathbf{J}_{wi} = \mathbf{z}_{i-1}$.

4.2 Column-by-Column Derivation

1. **Column 1 (Prismatic Joint 1):** $\mathbf{z}_0 = [0,0,1]^T \implies \mathbf{J}_{v1} = [0,0,1]^T, \quad \mathbf{J}_{w1} = [0,0,0]^T.$

2. **Column 2 (Revolute Joint 2):** $\mathbf{z}_1 = [0,0,1]^T, \quad \mathbf{p}_1 = [0,0,q_1 + 0.65]^T.$

$$\mathbf{J}_{v2} = \begin{bmatrix}0\\0\\1\end{bmatrix} \times \begin{bmatrix}-0.4\sin q_2 \cos q_3\\0.4\cos q_2 \cos q_3\\0.4\sin q_3 + 0.4\end{bmatrix} = \begin{bmatrix}-0.4\cos q_2 \cos q_3\\-0.4\sin q_2 \cos q_3\\0\end{bmatrix}, \quad \mathbf{J}_{w2} = \begin{bmatrix}0\\0\\1\end{bmatrix} \tag{17}$$

3. **Column 3 (Revolute Joint 3):** $\mathbf{z}_2 = [\cos q_2, \sin q_2, 0]^T, \quad \mathbf{p}_2 = [0,0,q_1 + 1.05]^T.$

$$\mathbf{J}_{v3} = \begin{bmatrix}\cos q_2\\\sin q_2\\0\end{bmatrix} \times \begin{bmatrix}-0.4\sin q_2 \cos q_3\\0.4\cos q_2 \cos q_3\\0.4\sin q_3\end{bmatrix} = \begin{bmatrix}0.4\sin q_2 \sin q_3\\-0.4\sin q_3 \cos q_2\\0.4\cos q_3\end{bmatrix}, \quad \mathbf{J}_{w3} = \begin{bmatrix}\cos q_2\\\sin q_2\\0\end{bmatrix} \tag{18}$$

Assembling the complete 6×3 Geometric Jacobian:

$$\mathbf{J}_G(q) = \begin{bmatrix}0 & -0.4\cos q_2 \cos q_3 & 0.4\sin q_2 \sin q_3\\0 & -0.4\sin q_2 \cos q_3 & -0.4\sin q_3 \cos q_2\\1 & 0 & 0.4\cos q_3\\0 & 0 & \cos q_2\\0 & 0 & \sin q_2\\0 & 1 & 0\end{bmatrix} \tag{19}$$

Why it is done: The Geometric Jacobian is essential for:

1. **Velocity Control:** Mapping desired Cartesian velocities to joint control commands ($\dot{q} = \mathbf{J}_G^{\dagger}\mathbf{v}$).

2. **Singularity Analysis:** Singularity configurations occur when $\det(\mathbf{J}_G\mathbf{J}_G^T) = 0$ (e.g., when $\cos q_3 = 0$), where the robot loses a degree of freedom and required joint speeds approach infinity.

3. **Statics & Force Control:** Via the principle of virtual work, $\boldsymbol{\tau} = \mathbf{J}_G^T\mathbf{h}_e$, mapping end-effector contact forces $\mathbf{h}_e$ to joint torques $\boldsymbol{\tau}$.

5 Analytical Jacobian (J_a)

5.1 Mathematical Formulation

What is being done: The Analytical Jacobian $\mathbf{J}_a(q) \in \mathbb{R}^{6\times3}$ is obtained by directly differentiating the operational pose vector $\mathbf{x} = [\mathbf{p}_e^T, \boldsymbol{\phi}^T]^T$ with respect to joint coordinates q:

$$\mathbf{J}_a(q) = \frac{\partial \mathbf{x}}{\partial q} = \begin{bmatrix}\frac{\partial \mathbf{p}_e}{\partial q}\\\frac{\partial \boldsymbol{\phi}}{\partial q}\end{bmatrix} \tag{20}$$

Where $\boldsymbol{\phi} = [\phi, \theta, \psi]^T$ represents end-effector orientation parameterized by ZYZ Euler angles extracted from the rotation matrix $\mathbf{R}(q)$:

$$\phi = q_2 + \frac{\pi}{2}, \quad \theta = q_3, \quad \psi = \frac{\pi}{2} \tag{21}$$

Differentiating $\mathbf{p}_e$ and ϕ yields:

$$\mathbf{J}_a(q) = \begin{bmatrix} 0 & -0.4\cos q_2 \cos q_3 & 0.4\sin q_2 \sin q_3 \\ 0 & -0.4\sin q_2 \cos q_3 & -0.4\sin q_3 \cos q_2 \\ 1 & 0 & 0.4\cos q_3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \tag{22}$$

Why it is done: Unlike angular velocity ω_e (which is a vector quantity that cannot be directly integrated into a position coordinate), operational orientation angles ϕ can be integrated. The Analytical Jacobian is therefore necessary for operational space control algorithms (e.g., Cartesian impedance control and PD pose tracking), where error feedback is expressed as a direct coordinate difference $\mathbf{e}_x = \mathbf{x}_d - \mathbf{x}$.

6 MATLAB Implementation & Console Outputs

6.1 Overview of the Script (`FirstAssignment_2.m`)

What is being done: The MATLAB script `FirstAssignment_2.m` implements the entire kinematic pipeline symbolically using the Symbolic Math Toolbox. It imports the URDF robot model, sets up interactive 3D graphics, computes DK matrices, evaluates closed-form IK equations, and derives both Jacobians. Numerical display is standardized using `vpa(..., 4)` to convert fraction artifacts into clean 4-digit decimals.

Why it is done: Symbolic automation guarantees mathematical rigor and eliminates transcription errors. Verifying symbolic results against numerical URDF visualization ensures that frame definitions, link offsets, and joint axes perfectly match the physical robot definition.

6.2 Verified Console Logs

Below are the exact console outputs generated by executing `FirstAssignment_2.m` in MATLAB:

```
--- Base Transformation Matrix ---
Matrix TB0 (World Frame -> Joint 0 Frame):
   0.0000   -1.0000         0         0
   1.0000    0.0000         0         0
        0         0    1.0000    0.6500
        0         0         0    1.0000
------------------------------------------------------------

--- Intermediate Transformation Matrices ---
Matrix T01 (Base -> Joint 1):
   0.0000   -1.0000         0         0
   1.0000    0.0000         0         0
        0         0    1.0000    q1 + 0.65
        0         0         0    1.0000
------------------------------------------------------------
Matrix T02 (Base -> Joint 2):
  -sin(q2)         0    cos(q2)         0
   cos(q2)         0    sin(q2)         0
         0    1.0000         0    q1 + 1.05
         0         0         0    1.0000
------------------------------------------------------------
```

```
Matrix T03 (Base -> Joint 3):
  -sin(q2)*cos(q3)   sin(q2)*sin(q3)   cos(q2)  -0.4*sin(q2)*cos(q3)
   cos(q2)*cos(q3)  -sin(q3)*cos(q2)   sin(q2)   0.4*cos(q2)*cos(q3)
           sin(q3)           cos(q3)         0   q1 + 0.4*sin(q3) + 1.05
                0                 0         0                 1.0000
------------------------------------------------------------

Matrix TBE Final (Base -> End-Effector):
   cos(q2)  -sin(q2)*cos(q3)   sin(q2)*sin(q3)  -0.4*sin(q2)*cos(q3)
   sin(q2)   cos(q2)*cos(q3)  -sin(q3)*cos(q2)   0.4*cos(q2)*cos(q3)
         0           sin(q3)           cos(q3)   q1 + 0.4*sin(q3) + 1.05
         0                0                 0                 1.0000
============================================================
```

Listing 1: MATLAB Console Output: Direct Kinematics Matrices.

```
--- Analytical Closed-Form Inverse Kinematics Solutions ---
Solution Branch 1:
q1 = pz - 0.4*sin(atan2((1.0 - (px^2 + py^2)/0.16)^(1/2), (px^2 + py^2)
   ^(1/2)/0.4)) - 1.05
q2 = atan2(-px, py)
q3 = atan2((1.0 - (px^2 + py^2)/0.16)^(1/2), (px^2 + py^2)^(1/2)/0.4)
------------------------------------------------------------
Solution Branch 2:
q1 = pz + 0.4*sin(atan2((1.0 - (px^2 + py^2)/0.16)^(1/2), (px^2 + py^2)
   ^(1/2)/0.4)) - 1.05
q2 = atan2(-px, py)
q3 = -atan2((1.0 - (px^2 + py^2)/0.16)^(1/2), (px^2 + py^2)^(1/2)/0.4)
------------------------------------------------------------
============================================================

--- Geometric Jacobian JG (6x3) ---
        0   -0.4*cos(q2)*cos(q3)    0.4*sin(q2)*sin(q3)
        0   -0.4*sin(q2)*cos(q3)   -0.4*sin(q3)*cos(q2)
   1.0000                      0           0.4*cos(q3)
        0                      0               cos(q2)
        0                      0               sin(q2)
        0                 1.0000                    0
============================================================

--- Analytical Jacobian Ja (6x3) ---
        0   -0.4*cos(q2)*cos(q3)    0.4*sin(q2)*sin(q3)
        0   -0.4*sin(q2)*cos(q3)   -0.4*sin(q3)*cos(q2)
   1.0000                      0           0.4*cos(q3)
        0                 1.0000                    0
        0                      0                1.0000
        0                      0                    0
============================================================
```

Listing 2: MATLAB Console Output: Inverse Kinematics Solutions and Jacobians.